

CLR(1) parsing table produces the more number of states as compare to the SLR(1) parsing.



Parul University

Faculty of Engineering & Technology

$\Rightarrow$ SLR(1) grammar

$\hookrightarrow$ If SLR(1) has no parse table has no conflicts.
SLR(1) parser is more powerful than LR(0) parser.

SLR(1)
LR(0)

* CLR(1) parsers : (canonical LR parsers)

CLR(1) parsing use the canonical collection of LR(1) items to build the CLR(1) parsing table where in LR(0) and SLR(1) use LR(0) items.

LR(1) item : LR(0) item + lookahead (LR(0), lookahead)

Ex:

- Gr:
- ① $S \rightarrow AA$
 - ② $A \rightarrow aA$
 - ③ $A \rightarrow b$

In CLR(1), the closure is found as, $\{ \}$
closure $(A \rightarrow \alpha \cdot \beta, x) = \{ A \rightarrow \alpha \cdot \beta, x, B \rightarrow \gamma, \text{First}(\beta x) \}$

For augment product, lookahead is $\$$

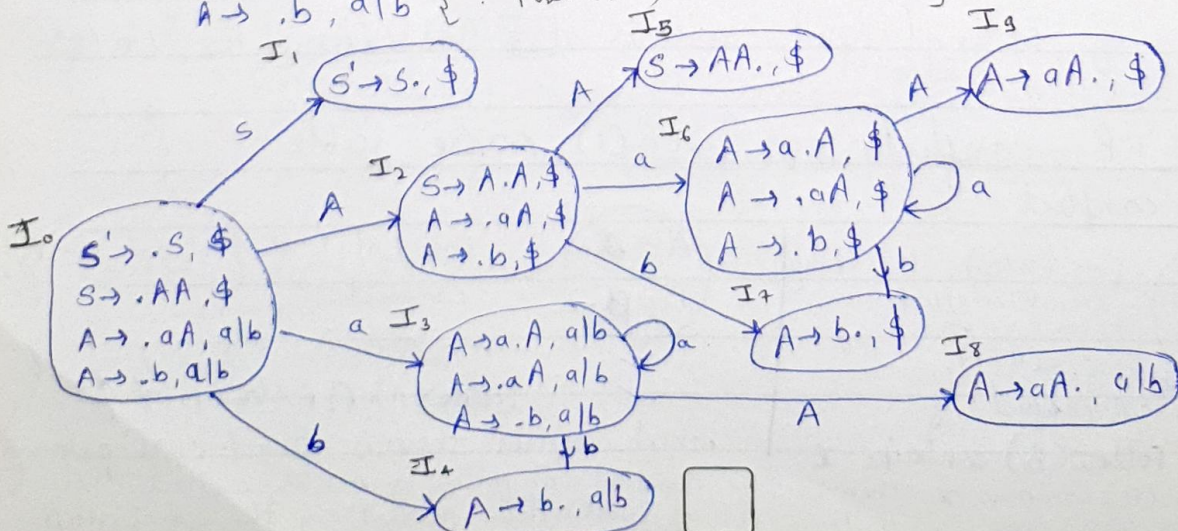
G' : after adding augment product, inserting $\cdot$ at beginning + product ϵ also adding a lookahead.

$S' \rightarrow \cdot S, \$$

$S \rightarrow \cdot AA, \$$ $\{ \because \text{First}(\epsilon \$) = \{ \$ \} \}$

$A \rightarrow \cdot aA, a/b$ $\{ \because \text{First}(a \$) = \text{First}(A) = \{ a, b \} \}$

$A \rightarrow \cdot b, a/b$ $\{ \because \text{First}(Aa) = \text{First}(A) = \{ a, b \} \}$





3. contains $S \rightarrow .AA, \$$ through which I_2 generated
 $\text{goto}(S \rightarrow .AA, \$) = \text{closure}(S \rightarrow A.A, \$)$

$= \{ S \rightarrow A.A, \$$ - First $S \rightarrow A.A, \$$ copied as it is
 $A \rightarrow .aA, \$$ - Now as after there A write LR(0)
 $A \rightarrow .b, \$$ } items of A keeping lookahead empty
 * Next shift the . one more time then take first i.e. $A.A \rightarrow AA, \$$ First (e\$ or

see the rule $\# \#$

	a	b	\$	S	A
I_0	S3	S4		1	2
I_1			Accept		
I_2	S6	S7			5
I_3	S3	S4			8
I_4	r3	r3			
I_5			r1		
I_6	S6	S7			9
I_7			r3		
I_8	r2	r2			
I_9			r2		

In LR(0), we write the reduce "op" in full row

SLR(1). " follow (LHS)

CLR(1), " lookahead.

Corresponding to state which consists of final items

* LALR parser : (Lookahead LR parser)

LALR works similar to CLR.

=> The only difference is, it combines the similar states of CLR parsing table into one single state. similar on the basis of LR(0) item (lookahead may differ)

In the above CLR example, we can combine

3 & 6 ; 4 & 7 ; 8 & 9



$$\underline{CLR(1) = LR(1)}$$



Parul University

Faculty of Engineering & Technology

LALR parse table

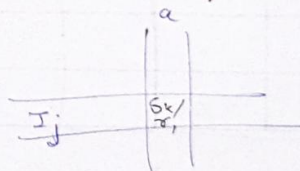
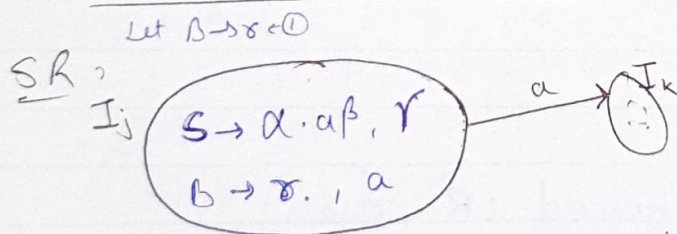
	a	b	\$	s	A
I_0	S36	S47		1	2
I_1			Accept		
I_2	S36	S47			5
I_{36}	S36	S47			89
I_{47}	r3	r3	r3		
I_6			r1		
I_{89}	r2	r2	r2		

- While combining the rows keep all the cells of the row which is being combined.
not covered. (ex: r3 of \$ + r2 of \$)

Note: #states in $LR(0) = SLR(1) = LALR(1)$

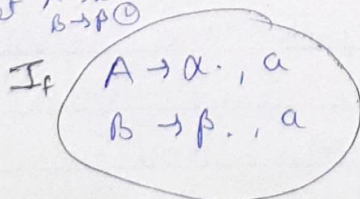
$$CLR(1) \geq LALR(1)$$

* Conflicts: SR & RR (CLR & LALR)

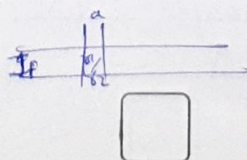


Hence, I_j on 'a' can shift to $I_k \Rightarrow S_k$
& can reduce with ① $\Rightarrow r1$ } SR conflict

RR: Let $A \rightarrow \alpha$ ①
 $B \rightarrow \beta$ ②



Here I_f can reduce with ① & ② both when lookahead is 'a' i.e. same



RR-conflict

$\Rightarrow$ CLR(1) grammar: If there is no SR or RR conflicts in CLR(1) parse table, then it is CLR(1) grammar.

Ex:

G: ① $S \rightarrow AaAb$

Augmented grammar
G:

② $S \rightarrow BbBa$

$\Rightarrow S' \rightarrow .S, \$$

③ $A \rightarrow d$

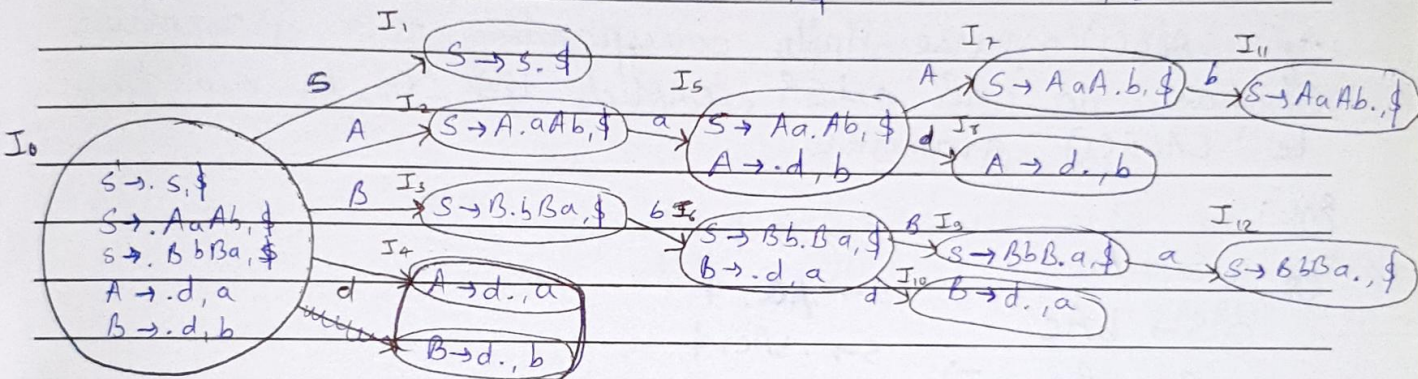
$S \rightarrow .AaAb, \$$

$A \rightarrow .d, a$

④ $B \rightarrow d$

$S \rightarrow .BbBa, \$$

$B \rightarrow .d, b$



- If we analyse each state carefully, we can get to know there isn't any SR or RR conflict $\Rightarrow$ CLR(1) grammar. Just to confirm & for practice let us construct parse table.

	a	b	d	\$	S	A	B
I ₀			S4		1	2	3
I ₁				Accept			
I ₂	S5						
I ₃		S6					
I ₄	r3	r4					
I ₅			S8			7	
I ₆			S10				9
I ₇		S11					
I ₈		r3					
I ₉	S12						
I ₁₀	r4						
I ₁₁				r1			
I ₁₂				r2			

Hence, from the parse table also it is confirmed that G is CLR grammar.



LALR(1) grammars and conflicts

(as we know LALR(1) parse table got generated by combining states in CLR(1) parse table, the following are some properties regarding conflicts of LALR)

$\Rightarrow$ If CLR(1) parse table has no SR conflicts then no SR conflicts in LALR(1) parse table

$\Rightarrow$ If CLR(1) parse table has no RR conflicts then there may/may not exist in LALR(1) table

If LALR(1) parse table corresponding to a grammar G has no RR and SR conflicts, then G is said to be LALR(1) grammar.

Ex:

Gr: ① $S \rightarrow Aa$

② $S \rightarrow bAc$

③ $S \rightarrow Bc$

④ $S \rightarrow bBa$

⑤ $A \rightarrow d$

⑥ $B \rightarrow d$

$S' \rightarrow .S, \$$

$S \rightarrow .Aa, \$$

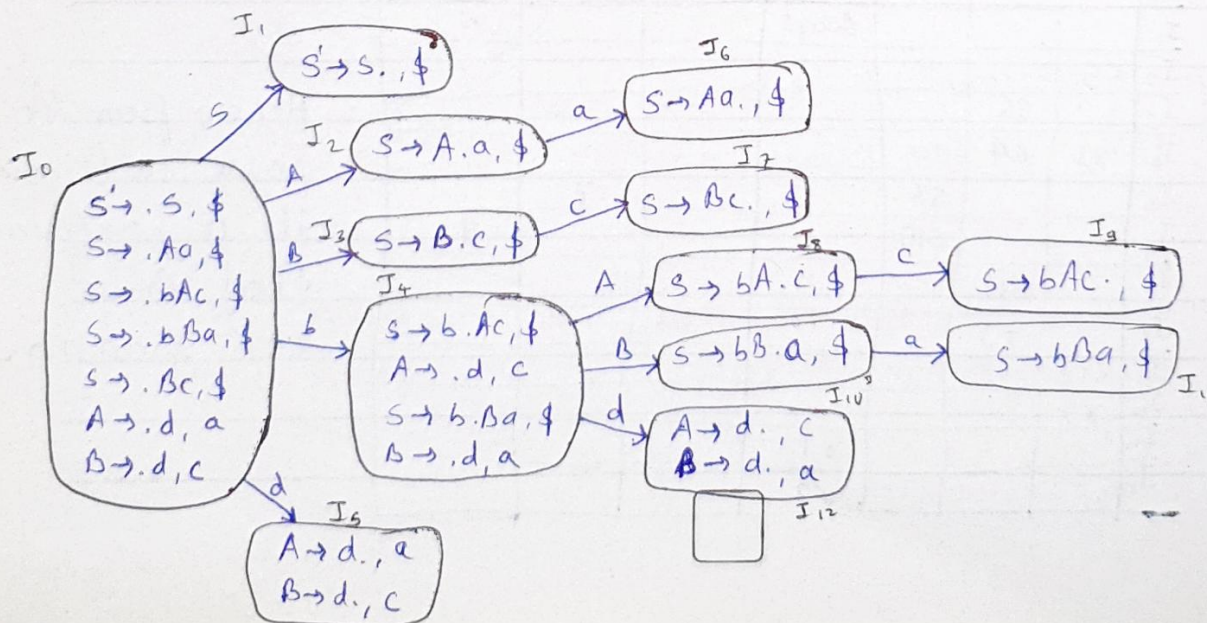
$S \rightarrow .bAc, \$$

$S \rightarrow .Bc, \$$

$S \rightarrow .bBa, \$$

$A \rightarrow .d, a$

$B \rightarrow .d, c$





For CLR(1)

From the graph, we can see that there isn't any SR or RR conflicts. There is two reduce operations which can take place from I_5 but ^(as it contains 2 final products) both ^{LR(0) items} productions have different lookaheads which implies no RR conflict. Similar for I_{12} . $\Rightarrow G$ is CLR(1) grammar

For LALR(1), as we know in LALR(1) we merge the states which contains similar LR(0) items.

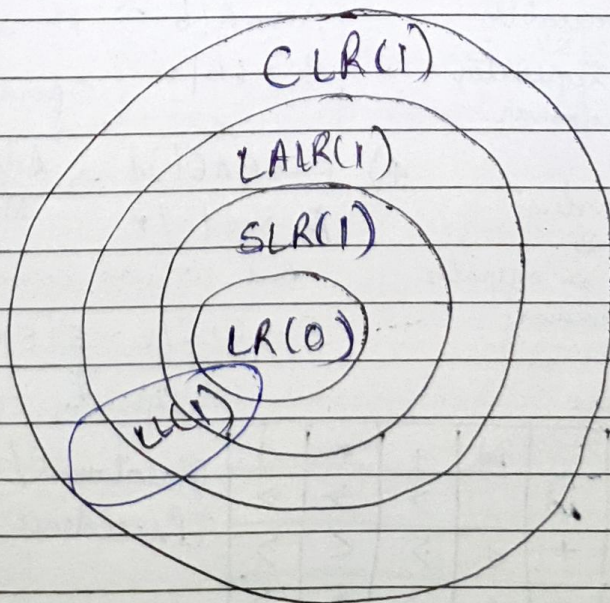
After merging I_5 and I_{12} (as it contains similar LR(0) item) we I_5, I_{12} which will contain

$A \rightarrow d, c|a$

$B \rightarrow d, c|a$

$\Rightarrow$ This will result into an RR conflict

$\Rightarrow G$ is not an LALR(1) grammar



$\Rightarrow$ CLR(1) parser is more powerful ^(in terms of parsing more grammars)

$\Rightarrow$ In practice ^(computationally) we use LALR(1) as it requires less (fewer) state and uses lookahead

* Operator Precedence Grammar

- ↳ Bottom Up Parser works for both ambiguous & unambiguous G.
- ↳ It can only parse a subset of CFG (i.e. it can't parse all CFG) which is called as operator grammar.

⇒ Operator Grammar

A grammar is said to be operator grammar if:

- In this, all terminals are operators.
- No R.H.S. of any production has ϵ (i.e. no ϵ -production)
- No two non-terminals are adjacent (i.e. AB is invalid)

- In operator grammar, all terminals are operators

Ex: 1) $S \rightarrow \widetilde{A}\widetilde{B}$

$A \rightarrow a$ adj. variable
 $B \rightarrow b$ NOT an operator grammar

2) $S \rightarrow AaB$

No ϵ -production
 $A \rightarrow aA|b \Rightarrow$ No adj. var.
 $B \rightarrow bB|a$ Operator G.

3) $S \rightarrow AaB$

$A \rightarrow aA|b \Rightarrow \epsilon$ -production
 $B \rightarrow bB|\epsilon$ NOT an operator grammar

4) $E \rightarrow EAE|id$

$\Rightarrow$ Adj. var.
 $A \rightarrow +|-|/*$ NOT an operator G.
But we can convert it as
 $E \rightarrow E+E/E-E/B * E/id$

Ex: G:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

$w = id + id * id \$$

	id	+	*	\$
id		>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

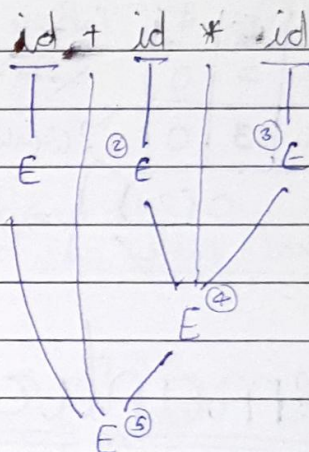
Operator G.
Relational / Precedence - table





An operator grammar which consists of only one entry or no entry in its precedence table is termed as operator precedence grammar.

Stack	Input	Action	Using precedence table
\$	id + id * id \$	Push	$< \Rightarrow$ Push/Shift
\$id	+ id * id \$	Pop + reduce ①	$> \Rightarrow$ Pop & Reduce
\$	+ id * id \$	Push	Empty $\Rightarrow$ Error
\$+	id * id \$	Push	
\$+id	* id \$	Pop + reduce ②	
\$+	* id \$	Push	
\$+*	id \$	Push	
\$+*id	\$	Pop + reduce ③	① After reducing id
\$+*	\$	Pop + reduce ④	
\$+	\$	Pop + reduce ⑤	
\$	\$	Accept	



- As we reached the starting symbol $\$$ stack, input both contains $\$$ only, the given w got accepted.
- The above is an ambiguous grammar which proves operator precedence grammar can also parse ambiguous grammars.
- One of the major drawback is the space complexity of precedence table, i.e. $O(n^2)$ for n operators.
- So, the ^{space} complexity of precedence table can be reduced from $O(n^2)$ to $O(2n)$ by converting relational table to functional table as:





Parul University

Faculty of Engineering & Technology

f {

	id	+	*	\$
id		>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

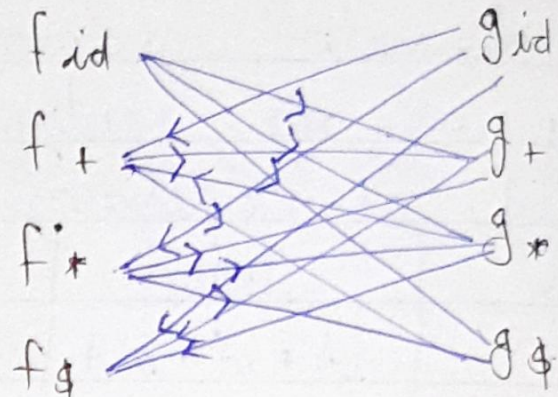
g

(Relational table)

$\triangleright \Rightarrow \rightarrow$
 $\triangleleft \Rightarrow \leftarrow$

$\Rightarrow$

!



longest path

Functional table

	id	+	*	\$
f	4	2	4	0
g	5	1	3	0

Functional table $O(2n)$

longest path size

$f_{id} \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_*$

$\Leftarrow$

$g_{id} \rightarrow f_* \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_*$

Working: If we want to compare f_+ & g_* $\Rightarrow 2 < 3 \Rightarrow$ shift/push $\Rightarrow$ can't find the errors i.e. empty cells.